GPS tracking and polygon geofencing — source audit

Date: 2026-10-17 Scope: `permission → location provider → updates → TrackingController → GeoFenceEngine → containment → Tracking screen`

This audit traces the complete chain in the existing code. It records **what is proven to work**, **what is actually broken**, and the evidence for each claim. No claim here is based on a device run: see "Device status" at the end.

1. The chain, as it exists

Step	File	Verdict
Manifest permissions	`app/src/main/AndroidManifest.xml`	Correct. `ACCESS_FINE_LOCATION` **and** `ACCESS_COARSE_LOCATION` (so "approximate only" is a legitimate grant), `FOREGROUND_SERVICE_LOCATION`, `foregroundServiceType="location"`, `POST_NOTIFICATIONS`.
Permission model	`location/LocationPermissions.kt`	Correct. `hasForegroundLocation` accepts fine **or** coarse, so an approximate-only grant is not treated as a denial.
Real provider	`location/FusedLocationSource.kt`	Correct shape. Cold `callbackFlow`, one `LocationCallback` per collection, removed in `awaitClose`; no fallback coordinate anywhere.
Duplicate / cancelled updates	`FusedLocationSource` + `TrackingController.launchLocationCollection`	Correct. `start()` cancels the previous job before launching; `callbackFlow` + `awaitClose` makes a second subscription structurally impossible. Proven by `TrackingPipelineTest."starting twice opens exactly one location subscription"`.

Step	File	Verdict
Coordinates reaching engine	`TrackingController.onReading`	Correct. Persists, then reads **enabled zones fresh from the database** each fix, then evaluates.
Lat/lon order	`GeoPoint(latitude, longitude)`, `PolygonCodec.encode`, `MercatorProjection.toGeoPoint`	Consistent everywhere — no transposition found.
Containment	`geofencing/PolygonGeometry.kt`, `PolygonContainmentService.kt`	Correct and well tested: ray casting is order-robust; the accuracy radius is used only to decide whether the test can be *trusted*, never as a stand-in for the outline.
Hysteresis / sticky state	`geofencing/GeoFenceEngine.kt`	Correct. `BOUNDARY` / `UNCERTAIN` never flip membership, so a bad fix cannot fabricate an alert.
State → UI	`TrackingSnapshot.zoneStatuses` → `TrackingScreen` "Containment right now"	Wired, but **invisible until a zone exists**, and there is no way to see *why* a verdict was reached.

2. Root causes found

RC-1 (first actual root cause) — nothing recovers when the OS state changes

`ui/viewmodel/TrackingViewModel.kt:172-176`:

```
/** Called when the user returns from system settings and the permission state may have ch
fun onResumed() {
    if (!snapshot.value.isTracking) return
    if (!settings.value.backgroundTrackingEnabled) return
}
```

The function's documented job is to re-check permission/provider state on resume. Its body has **no effect at all** — both statements return early, and there is no assignment, no call and no state change. `grep` confirms it also has **zero callers** anywhere in the app, and `MainActivity` installs no lifecycle observer.

Consequences, in the order a user hits them:

1. The user taps **Start tracking** with location switched off. `TrackingController.start()` refuses and records `LOCATION_DISABLED` (`TrackingController.kt:114-119`).
2. The user switches location on and returns to the app. Nothing re-checks, so tracking stays dead until the user works out that they must tap Start a second time.
3. Same for the permission dialog: if the grant lands after the screen was recreated, or if the user grants it from Settings, there is no recovery path.
4. If permission is **revoked** while tracking, nothing stops the pipeline or tells the user.

Also part of this defect: because a refused start sets `isTracking = false`, the snapshot cannot distinguish "the user never asked" from "the user asked and we could not start". The user's intent is lost, which is precisely why recovery is impossible.

RC-2 — `TrackingViewModel` reports success for a start the controller refused

`ui/viewmodel/TrackingViewModel.kt:121-125`:

```
trackingController.start(backgroundTracking = wantsBackground)
if (wantsBackground) { backgroundControl.start() }
_message.value = "Tracking started."
```

"Tracking started." is shown **unconditionally**, including when `start()` bailed out because location services are off. The user is told tracking started while no fix will ever arrive. This is failure possibility #1 from the review ("GPS disabled, or no fresh location fix") being *misreported*.

RC-3 — the foreground service can stop itself before it is ever commanded to start

`location/LocationTrackingService.kt:67-83`. `onCreate()` runs **before** `onStartCommand()`, and it collects `trackingController.snapshot`, a `StateFlow`. A `StateFlow` replays its current value to a new collector immediately. So:

```
statusJob = lifecycleScope.launch {
    container.trackingController.snapshot.collect { snapshot ->
        if (!snapshot.isTracking) { stopSelf(); return@collect }
        ...
    }
}
```

If the service is created at any moment when the controller is not yet tracking — the OS restarting it under `START_STICKY` after the process was killed, or any start that races ahead of `controller.start()` — the replayed `isTracking = false` makes `onCreate` call `stopSelf()` *before* `onStartCommand()` gets the chance to start tracking. The service silently dies and the pipeline never comes up.

RC-4 (diagnostics gap, item 6) — the pipe is invisible

`FusedLocationSource` records `lastError`, including from `onLocationAvailability` ("the provider reports no usable location right now"). That value is **never read by anything**: it is not in the snapshot and not on any screen. So when a device reports "no usable location", the app shows only the generic "Waiting for GPS fix…" and the cause is unknowable from the UI. There is likewise no screen showing permission precision, the provider state, the fix age, the accuracy, the current coordinates *and* the per-zone geofence verdict with the tolerances that produced it.

3. What is NOT broken (checked, with evidence)

- Polygon containment for valid polygons — inside / outside / on-edge / within tolerance / concave notch / too-few-vertices / invalid coordinate are all covered by `PolygonContainmentServiceTest` (13 tests) and pass.
- Duplicate subscriptions, single alert per crossing, uncertain fixes not fabricating alerts, mid-session zone creation — covered by `TrackingPipelineTest` (13 tests) and pass.
- Lat/lon ordering and the metre-frame projection — `PolygonGeometryTest`, `MercatorProjectionTest`.
- Simulated coordinates can never appear as GPS: readings carry `LocationSource`, and `TrackingPipelineTest."the gps pipeline only ever writes GPS rows"` asserts it.

4. Fixes applied

Each fix is the smallest change that removes the cause; the existing UI, the stored polygons, the offline database and the architecture are untouched.

F-1 (RC-1) — the app now recovers when the OS state changes

- New `location/TrackingRecovery.kt` — a **pure** rule, `trackingRecoveryFor(TrackingRecoveryInput)`, returning `NONE` / `START` / `STOP` / `REQUEST_PERMISSION` / `WAIT_FOR_LOCATION`. Permission outranks everything, then the device-wide location switch, then starting.
- `tracking/TrackingSnapshot.kt` — new `trackingRequested` field. A start that is refused now records **the request** as well as the refusal, so the obstacle can be re-checked later. `isTracking` keeps its old meaning ("the pipeline is running"), so nothing that reads it changes behaviour.
- `tracking/TrackingController.kt` — `start()` records the request *before* the location-service check; `stop()` clears it.
- `ui/viewmodel/TrackingViewModel.onResumed()` — implemented for real, and it now **has a caller**: `TrackingScreen` installs a `LifecycleEventObserver` that fires on `ON_RESUME`. Returning

from the permission dialog or from Settings now re-evaluates the facts and honours the pending request. The OS facts are passed in from the screen so the ViewModel still has no `Context`.

F-2 (RC-2) — the app no longer claims a start that did not happen

`TrackingViewModel.startTracking` now reads back the resulting snapshot and reports the outcome: "Tracking started.", or an explicit "Tracking did not start: location is switched off on the device …" which also tells the user that returning to the screen will start it. A disabled GPS can no longer be presented as a working one.

F-3 (RC-3) — the foreground service can no longer stop itself before it starts

`location/LocationTrackingService.kt` gained a `commandReceived` flag set in `onStartCommand`. The "stop when the pipeline is idle" rule only fires once a command has actually arrived, so the replayed pre-start value from `onCreate` can no longer kill the service. The user-initiated stop path is unchanged.

F-4 (RC-4) — the diagnostics the review asked for

- `tracking/GeofenceDiagnostics.kt` — pure builder producing, per enabled zone, the containment verdict, the remembered membership, the rendered status, the distance to the nearest edge, **the hysteresis margin that was actually in force** (entry or exit), the fix accuracy and the accuracy ceiling; plus the overall verdict and the zone that produced it. Every value is copied from the engine's own `ZoneStatus` — nothing is recomputed.
- `ui/components/TrackingDiagnosticsCard.kt` — renders it, with permission precision (precise / approximate-only / not granted), the location-service switch, pipeline state, the provider's own error string, the last fix's age and time, the coordinates, the accuracy and the fix count.
- `TrackingSnapshot.providerError` — the provider's verbatim complaint, recorded by `FusedLocationSource` and previously **read by nothing**, is now carried on the snapshot, refreshed on every fix and preferred over the generic pipeline error in the GPS status.

5. Verification

- **Full Android JVM suite at the time of this audit: 307 tests, 0 failures, 0 errors, 0 skipped, across 29 classes** (baseline before this round: 275 / 26). Counted from `app/build/test-results/testDebugUnitTest/TEST-*.xml`. *(The count moves each round; the current total after the future-risk round is 484 across 45 classes - see `docs/test_report.md`.)*
- **32 of those tests are new**, all passing:
- `geofencing/GeofenceWalkthroughTest` (9) — successive real-location updates pushed through the real engine: a 300 m drawn zone and a ten-step walk from 100 m outside to 40 m clear on

the far side, asserting the containment verdict, the sticky membership and the **status the screen renders** at every step; exactly one ENTRY and one EXIT; an undecided (accuracy-straddling) fix fabricating nothing; a zone narrower than the tolerance band never reading as inside; and a ±60 m fix at the centre of a 100 m zone reading as undecided rather than "inside dangerous area".

- `location/TrackingRecoveryTest` (12) — every branch of the recovery rule, including the exact case that was broken: permission missing, then granted, then recovered.
- `tracking/GeofenceDiagnosticsTest` (11) — the diagnostics report the engine's numbers verbatim, distinguish "no zones" from "zones that could not be tested" from "no fix yet", name the correct margin, and pick the zone that produced the banner.
- `:app:assembleDebug` → BUILD SUCCESSFUL.
- Pre-existing geofence coverage still green: `PolygonContainmentServiceTest` 13, `PolygonGeometryTest` 22, `GeoFenceEngineTest` 17, `TrackingPipelineTest` 13, `GpsStatusTest` 15.

6. Device status (unchanged, and it matters)

No physical Android device and no emulator were available. `adb devices` lists nothing. Every statement above is from reading the source and from JVM/Robolectric tests. Nothing in this document claims GPS was observed working on a phone.